발표자 대본

AI로 쇼핑몰 만들기 — 잘 묻고, 반드시 검증한다 · 3인 · 20분

> 슬라이드 번호는 발표_전체.pdf 가 아니라 **각 발표자 PDF 기준**입니다 (표지 다음 장이 1번).
> 시간은 목표치입니다. 합이 각자 배정 시간보다 짧게 잡혀 있으니 여유분으로 쓰세요.
> 이 대본은 템플릿입니다 — 본인 말투로 고쳐 쓰는 걸 전제로 썼습니다.

AI를 어떻게 썼나 — 발표자 1 · 8분

	슬라이드	시간	말할 내용
1	**무엇을 만들었나**	35초	한국형 온라인 쇼핑몰입니다. 고객몰, 관리자 화면, 백엔드까지 셋 다요. 숫자로는 이 정도입니다.
2	**오늘 할 얘기**	20초	근데 오늘 하려는 얘기는 "쇼핑몰을 만들었다"가 아닙니다. AI를 어떻게 써서 만들었나 — 이게 전부입니다. 여러분이 내일 바로 써먹을 수 있는 걸 가져가시게 하는 게 목표예요.
3	**★ 네 번 바뀌었다**	45초	먼저 결론부터 말씀드리면, 저희도 처음부터 잘 쓴 게 아닙니다. 7주 동안 AI 쓰는 방식이 네 번 바뀌었어요. 그리고 중요한 건 — 각 단계가 앞 단계가 깨져서 생겼다는 겁니다. 하나씩 보여드릴게요.
4	**1기 — 그냥 시켰다**	45초	1기는 그냥 시켰습니다. "상품 페이지 만들어줘." 돌아온 걸 보니 매번 구조가 다르고, 우리 프로젝트 규칙은 하나도 안 지키고, 로딩이나 에러 화면도 없었어요. 받아서 고치는 시간이 직접 짜는 시간보다 길었습니다. 여기서 방식을 바꿨어요.
5	**2기 — 프롬프트 5요소**	70초	그래서 팀이 공식을 정했습니다. 좋은 프롬프트엔 이 다섯 개가 다 들어갑니다. 맥락, 할 일 하나, 따를 패턴, 출력 형태, 그리고 검증 요구.
6	**같은 요청, 다른 결과**	45초	실제 비교입니다. (읽지 말고 짚기) 여기 보시면 맥락, 할 일 하나, 따를 패턴, 검증까지 다 들어가 있죠. 같은 요청인데 결과가 완전히 다릅니다.
7	**공식만으론 안 됐다**	40초	그런데 공식만으로는 안 됐어요. 프롬프트는 좋아졌는데, 크게 시키면 여전히 무너졌습니다. "주문 워크플로우 전체 만들어줘" 하면 코드는 나와요. 근데 어디가 틀렸는지 알 수가 없습니다. 결국 되돌리고 처음부터였어요. 덩어리가 크면 검증할 수가 없거든요.

	슬라이드	시간	말할 내용
8	**규칙 세 개**	**70초**	그래서 규칙 세 개를 붙였습니다. 첫째, 바로 구현하지 말고 접근을 먼저 제안하게. 30분짜리 삽질을 30초 대화로 막습니다. 둘째, 한 프롬프트에 한 기능만. 셋째, 타입을 먼저 확정하고 mock 으로 화면부터 만들었어요. 타입만 맞춰두면 서로 안 기다려도 되니까 병렬 개발이 됩니다.
9	**★ 3기 — 함정 심기**	**70초**	3기가 제일 실용적인데요. 저희가 한 번 겪은 실수를 그냥 넘어가지 않고, 다음 프롬프트에 경고로 박았습니다. (짚기) mock 에 랜덤이나 현재시각 쓰지 마라, 계산 필드를 단독으로 선택하지 마라, dev 서버 켠 채로 빌드하지 마라. 이거 전부 저희가 한 번씩 당한 겁니다. AI는 같은 함정을 매번 다시 밟거든요. 적어두고 붙였더니 안 밟더라고요.
10	**4기 — 문서 문제**	**40초**	4기는 문서입니다. AI에게 매번 프로젝트를 설명할 수 없으니 문서를 만들어 붙여줬는데, 70개가 되면서 문제가 생겼어요. 사람이 읽으려고 쓴 문서를 AI에게 주니까 오히려 결과가 나빠졌습니다. 비유랑 배경이 토큰만 잡아먹고 정작 타입 정의는 묻히거든요.
11	**★ 문서 4층 분리**	**70초**	그래서 독자별로 나눴습니다. 사람용, AI용, 운영용, 그리고 점검기록. AI용 문서는 아예 "이 문서를 읽는 AI는"으로 시작해요. 점검기록은 결과만 남기지 않고 다시 돌릴 수 있는 스크립트를 같이 뒀습니다.
12	**★ 네 번 바꾸고 나서**	**60초**	결과를 정리하면 이렇습니다. 앞의 두 줄은 솔직히 체감이에요. 근데 뒤의 두 줄은 결과물로 확인됩니다. 커스텀 모듈 27개가 지금도 전부 같은 구조고, 문서는 70개가 쌓였는데도 AI 답변 품질이 안 떨어졌어요.
13	**정리 · 연결**	**25초**	정리하면, 저희는 처음부터 잘 쓴 게 아니고 네 번 깨지고 네 번 바꿨습니다. 그리고 깨질 때마다 사람이 기억하는 게 아니라 프롬프트랑 문서에 규칙으로 남겼어요. 이 방식으로 실제 뭘 만들었는지는 ○○님이 이어가겠습니다.

발표 팁

- **이 파트는 "진화 과정"입니다.** 각 기(期)마다 "이래서 깨졌고 → 그래서 이렇게 바꿨다"를 한 호흡으로 말하세요. 방법론만 나열하면 남의 얘기로 들립니다.
- **프롬프트 예시는 읽지 마세요.** 화면에 띄우고 짚어주는 게 훨씬 낫습니다.
- 3기(함정 심기)가 가장 실용적인 파트입니다. 청중이 내일 바로 따라할 수 있는 게 여기예요.
- "AI가 대신 해줬다"는 뉘앙스는 피하세요. **"우리가 어떻게 시켰나"**가 핵심입니다.
- 시간이 모자라면 **'같은 요청, 다른 결과'** 한 장을 빼세요 — 5요소 슬라이드와 겹칩니다.

AI로 무엇을 만들었나 — 발표자 2 · 7분

	슬라이드	시간	말할 내용
1	**주문은 복잡하다**	—	주문이 생각보다 복잡해요. 구매 버튼 누르면 끝이 절대 아닙니다. 중간에 하나라도 실패하면 전부 되돌려야 하고요. 이런 건 바로 코드부터 뽑으면 안 됩니다.
2	**접근 먼저 프롬프트**	2분	그래서 앞에서 말한 "접근 먼저" 방식으로 물었습니다. (짚기) 마지막 진행 부분이 핵심이에요 — 바로 코드 짜지 말고 표로 제안하라고 했습니다.
3	**돌아온 건 흐름표**	—	코드가 아니라 흐름표가 왔어요. 덕분에 "여기 순서 이상한데?"를 저희가 짚을 수 있었습니다. 코드가 나온 뒤에 발견했으면 다 갈아엎어야 했을 겁니다.
4	**근거 요구**	1분 20초	흐름은 잡혔는데 걱정되는 게 있었어요. 동시 주문이랑 할인 순서요. "안전합니다"라는 답만 받았으면 넘어갔을 텐데, 근거를 요구하니까 실제 코드를 확인하고 답하더라고요.
5	**한 프롬프트 = 한 기능**	—	처음엔 "주문 워크플로우 전체 만들어줘"라고 했다가 실패했어요. 코드는 나오는데 어디가 틀렸는지 알 수가 없었습니다. 너무 크니까요.
6	**쪼갰습니다**	2분	그래서 넷으로 쪼갰어요. 재고, 할인, 적립금, 그리고 합치기. 각 단계마다 테스트하고 넘어갈 수 있으니 훨씬 빨랐습니다.
7	**1단계 프롬프트**	—	이게 1단계 프롬프트예요. "다른 단계는 아직 X"를 명시했고, 따를 패턴을 지목했고, 검증까지 요구했습니다.
8	**기존 패턴 지목**	1분 20초	상품·회원 쪽에선 기존 코드를 지목하는 기법을 많이 썼습니다. "알아서 잘"이라고 하면 매번 다른 구조가 나와요. "저것처럼"이라고 지목하면 일관성이 유지됩니다. 그 결과 모듈 27개가 전부 같은 구조예요.
9	**★ 고치다 깨뜨렸다**	60초	여기까지만 들으면 술술 풀린 것 같은데 아닙니다. 고치는 과정에서 다른 걸 깨뜨린 적이 여러 번 있어요. 보안 취약점 대응으로 라이브러리 버전을 고정했더니 린터가 아예 안 돌았고, 재고 강제 차감을 쿼리 파라미터로 만들었더니 프레임워크 검증에 걸려서 헤더 방식으로 다시 짰고, 이미지 최적화를 켰더니 등록 안 된 이미지 호스트 때문에 상품 상세가 500이 났습니다. 셋 다 실제로 돌려보고 나서 알았어요. 그래서 "끝나면 검증까지"가 프롬프트에 들어간 겁니다.

	슬라이드	시간	말할 내용
10	**정리 · 연결**	**20초**	정리하면 네 가지입니다. 그런데 이렇게 만들었는데, AI가 만든 게 진짜 맞는지는 또 다른 문제였습니다. 검증 얘기는 ○○님이 이어가겠습니다.

발표 팁

- 발표자 1의 **5요소·2단계 진행법을 계속 인용**하세요. "앞에서 말한 그 방식으로" 라는 말을 반복하면 발표가 하나로 묶입니다.
- 프롬프트 블록은 **전문을 읽지 말고** 핵심 한 줄만 짚으세요 ("바로 코드 짜지 말고").
- "쪼갰다"가 이 파트의 메시지입니다. 결과물 자랑보다 **왜 쪼갰는지**에 시간을 쓰세요.
- 시간이 모자라면 상품·회원 슬라이드를 빼세요. 주문 사례만으로 충분합니다.

AI 결과를 어떻게 검증했나 — 발표자 3 · 7분

	슬라이드	시간	말할 내용
1	**AI는 그럴듯하게 틀린다**	**40초**	앞에서 AI로 빠르게 만든 얘기를 들으셨는데요, 저는 반대 얘기를 하겠습니다. AI는 그럴듯하게 틀립니다. 코드는 돌아가고 테스트도 통과해요. 그런데 틀렸습니다.
2	**검증 3층**	—	그래서 검증을 세 층으로 깔았습니다. 타입체크, 자동 테스트 960개, 그리고 보안 프로브 52건. 이 중에서 세 번째가 진짜를 잡았습니다.
3	**① 적립금 3배**	**1분**	적립금 코드는 "동시 요청에 안전하다"고 되어 있었어요. 락도 걸려 있었고요. 그런데 실제로 던져봤습니다. 장바구니 3개에 동시에 써봤더니 — 세 건 다 통과했습니다. 10,000원인데 30,000원어치 할인됐어요. 잔액은 마이너스 2만원.
4	**원인**	—	원인은 락을 잘못된 단위로 잡은 거였어요. 락 키가 장바구니별인데, 적립금은 고객별 자산이거든요. 장바구니를 여러 개 만들면 서로 다른 락을 잡으니까 의미가 없었던 겁니다. 주석에는 "동시 차감 방지"라고 적혀 있었어요.
5	**② 환불 3배**	**1분**	환불도 같은 방식으로 테스트했습니다. 같은 요청번호로 동시에 3번 보냈더니 3건 다 성공했어요. 1,000원 요청했는데 3,000원이 나갔습니다. 이건 더 무서운 게, 순차로 보내면 정상적으로 막혀요. 동시에 보낼 때만 뚫립니다.

	슬라이드	시간	말할 내용
6	**같은 원인**	**30초**	세 건 모두 이 패턴이었습니다. 검사하고, 집행하는 사이에 락이 없어서 다른 요청이 끼어드는 거죠. 고친 뒤 같은 공격을 다시 던져서 막히는 걸 확인했습니다.
7	**★ 저희 점검도 틀렸다**	**60초**	여기서 솔직하게 말씀드릴 게 있는데요. 저희 점검도 틀렸습니다. 방금 그 적립금 버그, 사실 일주일 전 점검에서 통과 처리했던 항목이에요. 주석에 "이중차감 방지"라고까지 적어놨는데 락 단위가 틀렸던 겁니다. 환불은 고쳤는데 취소에는 반영을 안 한 것도 있었고요. 심지어 "화면이 안 바뀌면 고장"이라는 판정 기준을 썼다가 멀쩡한 기능 4개를 고장으로 잘못 표시하기도 했습니다. 그래서 판정 기준을 항목별로 다시 잡았어요.
8	**③ 소셜 로그인**	**1분 30초**	돈 문제만 있었던 게 아닙니다. 카카오 로그인 버튼이 있었는데 실제로는 데모용 가짜였어요. 브라우저 두 개로 로그인해 보니 같은 계정에 들어갑니다. 이대로 열었으면 고객 B가 고객 A의 주문내역을 그대로 봤을 겁니다.
9	**검증을 자산으로**	**1분**	여기서 한 가지 더 한 게 있는데요. 찾은 걸 문서로만 남기지 않았습니다. 다시 돌릴 수 있는 스크립트로 만들었어요. 명령 한 줄이면 지금도 확인됩니다.
10	**★ 숫자로 본 결과**	**60초**	지금까지 얘기한 걸 숫자로 보면 이렇습니다. (한 줄씩 짚기) 여기서 마지막 줄을 봐주세요 — 백엔드 취약점 4건은 일부러 안 고쳤습니다. 실제로 도달할 수 없다는 걸 측정해서 확인하고, 왜 안전한지 문서에 남겼어요. 전부 0으로 만드는 것보다 왜 남겼는지 설명되는 게 중요하다고 봤습니다.
11	**마무리 1**	**—**	AI로 빠르게 만들 수 있었습니다. 그건 사실이에요. 근데 검증하지 않았으면 돈이 새는 걸 몰랐을 겁니다. 코드만 읽으면 안전해 보였고, 주석에도 방지함이라고 적혀 있었어요.
12	**마무리 2**	**20초**	잘 묻고, 반드시 검증한다. 저희가 배운 건 이거였습니다. 감사합니다.

발표 팁

- **"저희 점검도 틀렸습니다"가 이 파트의 신뢰를 만듭니다.** 남의 코드만 까면 방어적으로 들리는데, 자기 실수를 먼저 인정하면 나머지가 다 믿깁니다.
- **숫자를 말로 하세요.** "10,000원인데 30,000원어치 할인됐습니다" — 이게 제일 꽂힙니다.
- 적립금·환불 사례가 핵심입니다. **시간의 절반을 여기 쓰세요.**

- "AI가 잘못했다"보다 **"검증 안 하면 아무도 모른다"**로 프레이밍하세요. AI 비판이 아니라 검증의 중요성이 메시지입니다.
- 소셜 로그인 사례는 개인정보 얘기라 반응이 큽니다. 천천히 말하세요.